Technical Assessment Architecture Analysis

https://gemini.google.com/app/2d61ed0e6d3b09d7

User prompt: I have attached a techinal assesment for the role in the JD file. Let's start by analysing the problem by summarizing what is given and what is required.

Response: 1. Core Problem & Domain Context

Caseware builds platforms for audit, assurance, risk, and compliance professionals where AI is used to augment practitioner judgment—helping users understand context, investigate findings with traceability, and apply consistent methodologies—rather than replacing human decision-making.

Within this ecosystem, the assessment asks you to solve a metadata synchronization and change-summarization bottleneck across two core domain entities:

- **Product Templates:** Shared zip archives of structured JSON data maintained by content teams across different markets. They are updated relatively infrequently, around once per week per product.
- **Engagement Files:** Customer-specific workspaces containing a full year of accounting/audit work for a client. Each file is initialized from the latest Product Template version at creation time, and a typical firm manages hundreds of engagements across multiple products.

The User Need: When a new Product Template version is published, non-technical practitioners need to see **at a glance** which of their hundreds of Engagement Files have pending updates, view a **human-readable summary** of the inbound changes, and decide whether to **apply or decline** the update for each engagement.

2. What is Given (Existing Systems & Constraints)

System / Aspect	Provided Capabilities & Architecture	Key Constraints & Bottlenecks
Product Template System	Stores all versions of all templates in a single database shared across all customer firms. Supports fast lookups by `template ID` + `version`.	Has zero visibility into or retention of customer Engagement Files.
Engagement Management System	Stores Engagement Files in isolated, customer-specific (multi-tenant) databases. Handles engagement creation, loading, and processing the user's apply/decline action.	**Hard Constraint:** Loading an Engagement File takes **~1 minute per file**, and loading is currently required to read the `template ID` and `version` stored inside it.
Diffing Engine	A fast, reliable mechanism already exists to compare any two versions of a Product Template and extract a **raw JSON diff**.	Raw JSON diffs are unsuitable for non-technical audit professionals who require human-readable summaries.
Integration Hooks	You can attach **hooks or domain events** to any action in both the Product Template System (e.g., template published) and the Engagement Management System (e.g., engagement created, update accepted/declined).	Applying the actual template content to the Engagement File is **out of scope**.
Target Tech Stack	You may choose any stack, though Caseware is an AWS shop. The role's stack uses **Java 25, Spring Boot, Spring Cloud Gateway**, **AWS (S3, SQS, SNS, Lambda, DynamoDB, DocumentDB, PostgreSQL, EKS)**, **Terraform**, and **New Relic**.	Expected effort is time-boxed to **2–3 hours**, prioritizing clarity and architectural trade-offs over over-optimization.

3. What is Required (System Requirements & Key Challenges)

To satisfy the prompt, your architecture must solve four specific engineering problems:

1. **Decoupling Metadata for Fast Reads ("At-a-Glance" View):** Because reading an engagement file takes ~1 minute, you cannot query the Engagement Files directly when a user opens their dashboard. You need a lightweight, event-driven metadata projection (read model) that tracks `(firm_id, engagement_id, product_id, current_template_version, last_evaluated_version)` via hooks emitted on engagement creation and update resolution.
2. **Human-Readable Change Summarization:** You need a pipeline that takes the raw JSON diff between template versions and generates a clear, accurate, non-technical summary (using an LLM with strict grounding/validation) so practitioners can confidently make an apply/decline decision.
3. **Handling Accumulated Updates (`v1 -> v2 -> v3`):** You must define how the system behaves when multiple weekly updates occur before a user logs in and acts. This includes deciding whether diffs and summaries are pre-computed pairwise (`v1->v2`, `v2->v3`) or dynamically/cumulatively (`v1->v3`), and defining the state transitions if a user declines `v2` before `v3` arrives.
4. **Multi-Tenant Scale & Efficiency:** Since Product Templates are shared globally across firms while Engagement Files are tenant-isolated, diff extraction and LLM summary generation between `(template_id, from_version, to_version)` should ideally be computed once and cached/shared across all firms rather than re-computed per engagement file.

4. Deliverables & Evaluation Criteria

Part 1: Architecture & Design Document (Primary Focus — 1 to 3 pages max)

Submitted as PDF or Markdown along with any architecture diagrams, covering five mandatory sections:

1. **High-Level Architecture:** Service boundaries, event hooks, data stores, caching, and LLM summarization pipeline.
2. **Implementation Plan:** How to break the initiative into parallelizable, incremental milestones that deliver business value.
3. **Testing Strategy:** Unit, contract, integration, and deterministic vs. non-deterministic (AI output) validation.
4. **Evaluation & Observability:** System health metrics (SLIs/SLOs, queue depth, latency via tools like New Relic) and AI summary quality/hallucination monitoring.
5. **Failure Modes & Tradeoffs:** Out-of-order/missed events, eventual consistency gaps, LLM downtime or hallucinations, and cost vs. latency trade-offs in cumulative diff generation.

Part 2: Targeted Implementation (Optional, Strongly Encouraged)

- A focused code slice (ideally in **Java / Spring Boot** to match the JD) emphasizing clean **interfaces, domain contracts, state transitions, and unit tests/validation logic** rather than boilerplate infrastructure.

AI Usage & Follow-Up Review Readiness

- Both the prompt and the JD heavily emphasize **AI-augmented engineering maturity**. In the follow-up interview, you will be asked to defend:
  - Where AI accelerated your work and where you **corrected or rejected** AI suggestions.
  - How you would mentor other developers on using AI tools on this system.
  - **Where AI must not be trusted in this domain** (e.g., deterministic version tracking, tenant isolation, and ensuring LLM summaries never fabricate or omit compliance-critical changes).

Would you like to start by working through the **core architectural decisions and edge cases** (such as how to model accumulated updates and decline states), or jump straight into structuring **Part 1 (High-Level Architecture)**?

User prompt: Based on these constraints, here is my initial architectural direction. Review it and point out any edge cases or possible bottlenecks:1. Decoupling from the 1-minute file load: we can attach hooks/events to the Engagement Management System and Product Template DB, so I want to introduce an event-driven Read Model. I am thinking it should be called Engagement Lightweight Store in PostgreSQL or maybe DynamoDB as the company uses AWS. When an engagement is created, or an update is applied/declined, an event updates this lightweight store with `(engagement_id, firm_id, template_id, applied_version, declined_version)`. 2. Caching diffs/summaries at the Template level: Thousands of engagements across firms share the same Product Templates. Generating LLM summaries per engagement or on-demand when a user opens the UI would be expensive and slow. Instead we should do an asynchronous compute on the JSON diff. When a new template version `v_new` is published, the LLM should summarize once per `(template_id, v_from, v_new)` pair. That is for all currently active versions `v_from` in our metadata store. 3. Handling accumulated updates & declines: An engagement always transitions from `applied_version` to `latest_version`. If user is on `v1`, declines `v2`, and later `v3` is published, the user should be notified again for `v3`. The summary shown must be the cumulative diff `(v1 -> v3)`, not `(v2 -> v3)`, because `v2` was never applied. How well does this hold up against the requirements?

Response: Your architectural direction hits all three core engineering problems in the prompt: it bypasses the 1-minute engagement load bottleneck, avoids redundant per-tenant LLM calls, and correctly models state transitions across declined versions.

To make this design airtight for a Senior-level review, here is how each pillar holds up against edge cases, multi-tenant boundaries, and operational bottlenecks.

1. Event-Driven Read Model (`Engagement Lightweight Store`)

What works well: Projecting `(engagement_id, firm_id, template_id, applied_version, declined_version)` into a lightweight read store completely decouples the dashboard "at-a-glance" read path from the 1-minute Engagement File load time.

Edge Cases & Bottlenecks to Address:

- Read-Time Join vs. Write-Time Fan-Out (PostgreSQL vs. DynamoDB):Both PostgreSQL and DynamoDB are part of Caseware's AWS stack, but they change how you determine whether an engagement has a pending update:
  - **PostgreSQL (Recommended for this access pattern):** You can store a small `product_template_latest` table `(template_id, latest_version, published_at)` alongside the `engagement_metadata` table in the same database. When a `TemplatePublished` event arrives, you do **1 write** to `product_template_latest`. When a user opens their dashboard, a single indexed query joins the firm's ~100s of engagements against `product_template_latest` where `latest_version > applied_version AND (declined_version IS NULL OR latest_version > declined_version)`.
  - **DynamoDB:** Because there are no joins, publishing a template either requires fanning out writes to every affected engagement across all firms (write amplification), or doing an application-side join in your Spring Boot service (fetching the firm's ~100 engagements from DynamoDB and matching against an in-memory/cached map of latest template versions).
- **In-Flight State During the 1-Minute Apply Operation:** Applying an update requires loading and mutating the Engagement File (~1 minute). If a user clicks "Apply," the read model needs an intermediate state (e.g., `status = APPLY_IN_PROGRESS` or `PENDING_DECISION_PROCESSING`)

before the `UpdateApplied` event returns. Otherwise, another practitioner in the same firm could click "Apply" or "Decline" concurrently while the file is still loading.

- **Cold Start / Legacy Backfill:** Event hooks only capture *new* creations and decisions going forward. In your design doc, briefly state how existing engagements are populated—either via a throttled, off-peak background migration job (respecting the 1-minute load constraint) or via lazy backfill whenever an existing engagement is loaded by a user.
- **Out-of-Order Events & Idempotency:** If message queues (e.g., AWS SQS/SNS) deliver an older event after a newer one, the read store could regress an engagement's version. Enforce optimistic locking (`version_sequence` or `event_timestamp`) so stale events are safely ignored.

2. Caching Diffs & Summaries at the Template Level

What works well: Since Product Templates are shared globally across all customer firms, computing the summary per `(template_id, v_from, v_new)` rather than per engagement reduces LLM invocations by orders of magnitude and keeps LLM latency off the user's synchronous read path.

Edge Cases & Bottlenecks to Address:

- **Cross-Boundary Querying for Active `v_from` Versions:** The Product Template system does not retain any information about Engagement Files, which live in customer-specific databases. For the Summarization Service to know which `v_from` versions are currently "active," it must query your new Read Model (e.g., `SELECT DISTINCT applied_version FROM engagement_metadata WHERE template_id = :id`) without leaking tenant-identifiable data.
- **Context Window Overflow on Long-Tail Versions (`v1 -> v25`):** Templates update roughly once per week. Over 6 months, an inactive engagement could be 25 versions behind. Feeding a raw cumulative JSON diff of `v1 -> v25` into an LLM creates two risks:
  1. The raw JSON diff may exceed the LLM context window or trigger "lost in the middle" degradation where critical audit changes are omitted.
  2. Pre-computing every `(v_from, v_new)` pair using massive raw diffs becomes progressively slower and more expensive.
- **Refinement — Two-Tier / Hierarchical Summarization:**
  - Whenever `v_new` is published, immediately compute the **single-step diff and summary** `(v_new - 1 -> v_new)`.
  - For cumulative spans `(v_from -> v_new)`, if the raw JSON diff `(v_from -> v_new)` is below a token threshold, summarize the raw cumulative diff directly (which cleanly cancels out intermediate reverts, e.g., a field added in `v2` and removed in `v3`). If it exceeds the threshold, synthesize the cumulative executive summary from the validated step-by-step summaries alongside deterministic JSON change metadata.
- **Race Condition on Newly Published Templates:** If a user opens their dashboard immediately after `v_new` is published—before the async LLM worker finishes—the UI shouldn't hide the pending update or fail. Return the pending update banner immediately with `summary_status: GENERATING` and let the UI poll or receive a push once the summary is cached.

3. Handling Accumulated Updates & Declines (`v1 -> v2 (declined) -> v3`)

What works well: Your state rule is technically sound: declining `v2` does not mutate the engagement file (which stays on `v1`), and publishing `v3` must re-notify the user because `latest_version (v3) > declined_version (v2)`. Furthermore, because templates are complete zip archives of structured JSON data, applying `v3` to a `v1` engagement applies the full state of `v3`, making `(v1 -> v3)` the true net change to the engagement file.

Edge Cases & Nuances to Address:

- **The Practitioner UX / Audit Confusion Trap:** Suppose an auditor is on `v1`, carefully reviews the `v1 -> v2` summary, and declines `v2`. A week later, `v3` arrives. If the UI *only* shows a monolithic `(v1 -> v3)` summary without distinguishing what changed in `v3`, the auditor faces two problems:
  1. They have to mentally filter out the `v2` changes they already reviewed last week just to see what is new in `v3`.
  2. Conversely, they must be explicitly warned that accepting `v3` is an all-or-nothing upgrade to the `v3` template state—meaning it **will include** the previously declined `v2` changes (unless `v3` reverted them).
- **Refinement — Dual-Context Summary Payload:** Because your read model tracks *both* `applied_version` (`v1`) and `declined_version` (`v2`), your API can return:
  - **Total Impact (`v1 -> v3`):** The full cumulative summary of what will change in the engagement file if applied.
  - **New Since Last Review (`v2 -> v3`):** The delta since the version they declined, giving the practitioner immediate clarity on why they are being prompted again.

4. Crucial Addition for the Rubric: Deterministic Guardrails Around AI

The assessment explicitly notes that Caseware uses AI to augment professional judgment with **traceability and confidence**, and that you will be asked **where AI should not be trusted in this domain**. Make sure your design includes:

Component	Deterministic (Never Trust AI)	AI-Augmented (LLM Role)
Version & State Tracking	Determining whether an update is pending (`v_latest > v_applied`) and which files are affected.	None—strictly handled by relational/state logic.
Diff Extraction & Counts	Extracting added/modified/deleted JSON paths and counting affected sections/forms via the existing JSON diff engine.	Translating technical JSON keys/values into plain-language audit/accounting descriptions.
Hallucination / Omission Guardrails	Programmatically verifying that every modified top-level section in the JSON diff is referenced in the structured summary output, with a fallback to a deterministic structured table if validation fails.	Grouping low-level JSON field edits into readable bullet points cited back to specific template section IDs.

Would you like to lock in the AWS technology choices (e.g., PostgreSQL + SQS/SNS + Spring Boot) and start drafting the **1–3 page Design Document (Part 1)**, or refine the data model and state machine first?

User prompt: I agree with using PostgreSQL as the Read Model to avoid DynamoDB write fan-outs. Adding the intermediate `APPLY_IN_PROGRESS` state is a great catch to prevent concurrent mutations during that 1-minute window. However, two important constraints I want to enforce before we write the document:1. We should NOT ask the LLM to synthesize/combine multiple text summaries together (ex: combining the text summary of v1->v2 with v2->v3) to handle accumulated updates. That introduces compounding LLM hallucinations and fails when a change in v2 is reverted in v3. Instead, as long as the raw JSON diff (v1 -> v3) is within token limits, we must always run the JSON Diff tool directly between `(applied_version, latest_version)` and pass that exact diff to the LLM. 2. In an audit and compliance domain, we cannot trust raw LLM output blindly. Let's design a strict guard: the LLM must output structured JSON referencing the exact paths/sections from the JSON diff. If the LLM fails validation or times out, the UI must degrade to showing a structured, human-formatted view of the diff headers, rather than blocking the user. Let's now generate the complete 1-3 page Architecture Design Document in Markdown following the 5 required sections (High-Level Architecture, Implementation Plan, Testing Strategy, Evaluation & Observability, Failure Modes & Tradeoffs). Please include Mermaid.js code blocks for: - The High-Level Event-Driven Architecture diagram. - The State Machine diagram for Accumulated Updates & Decline.

Response: I will now generate the **Architecture Design Document** in Markdown format, incorporating the PostgreSQL read model, the `APPLY_IN_PROGRESS` concurrency lock, direct net-diff computation across accumulated versions, strict deterministic AI validation with graceful fallback, and the Mermaid architecture and state machine diagrams.

I have created the complete **Architecture Design Document** (`architecture_design_document.md`) covering all five required sections, the Mermaid architecture and state machine diagrams, and your strict constraints around direct net-diffing and deterministic AI guardrails.

Would you like to refine any section of this document, or move on to **Part 2 (Targeted Implementation)**—for example, implementing the core Spring Boot / Java 25 state transition engine and the deterministic LLM Citation & Coverage Validator with JUnit tests?

User prompt: The design document looks solid. Now let's implement Part 2 (Targeted Implementation) in Java 25 and JUnit 5. Keep it focused strictly on domain contracts, state transitions, and the deterministic AI validator from Sections 1.3 and 1.4 (no full Spring Boot/DB wiring needed): 1. Domain records/models for `EngagementMetadata` (including `appliedVersion`, `declinedVersion`, and `APPLY_IN_PROGRESS` status), `TemplateDiff`, and `AiSummaryResult`.2. Interfaces for `TemplateDiffEngine` and `LlmSummarizer`. 3. `SummaryCitationValidator` & `SummarizationOrchestrator` checking that LLM output cites only valid diff paths and covers 100% of modified sections, falling back to the structural diff on failure/timeout. 4. `EngagementUpdateDomainService` handling pending update checks, accumulated updates (v1 -> v2 -> v3), declining v2 then receiving v3, and concurrency locking. 5. JUnit 5 tests covering these scenarios.

Response: Your ZIP file is ready

[file-tag: code-generated-file-7391d995-d49f-4140-a441-bbe0bb4b0818]

Individual source files are also available for direct download:

- `pom.xml`: [file-tag: code-generated-file-ed6df9dc-857f-4f67-841b-9d32d5c7d6aa]
- `DomainModels.java`: [file-tag: code-generated-file-b3e6d68c-454f-42ce-a22e-d61d00a5049c]
- `Ports.java`: [file-tag: code-generated-file-3328a8f4-50c1-4199-b318-9ff93f4a9348]
- `SummaryCitationValidator.java`: [file-tag: code-generated-file-4cf23968-c4b8-4ba7-8715-5f24f7c1d503]
- `EngagementUpdateDomainService.java`: [file-tag: code-generated-file-701357ce-5e70-4897-a3b1-a7621a28a507]
- `EngagementUpdateSystemTest.java`: [file-tag: code-generated-file-8e1e2557-60b7-42ca-86d9-1e6c18f995fc]

Below is the complete **Part 2 (Targeted Implementation)** in **Java 25** and **JUnit 5**, designed around immutable domain records, explicit port contracts, deterministic AI citation/coverage validation, and concurrency-safe state transitions.

1. Domain Records & State Transitions (`DomainModels.java`)

Encapsulates `EngagementMetadata` (including `appliedVersion`, `declinedVersion`, `APPLY_IN_PROGRESS` locking, and `eventSequence` idempotency guards), `TemplateDiff`, and `AiSummaryResult`.

```
package com.caseware.updates;

import java.util.List;
import java.util.Objects;
import java.util.OptionalInt;
import java.util.Set;
import java.util.UUID;
import java.util.stream.Collectors;

/**
 * Core domain models and records for Engagement Metadata, Template Diffs, and AI Summaries.
 */
public final class DomainModels {

    private DomainModels() {}

    public enum EngagementStatus {
        READY,
        APPLY_IN_PROGRESS
    }

    public enum ChangeType {
        ADDED,
        MODIFIED,
        REMOVED
    }

    public enum SummarySource {
        AI_VALIDATED,
        DETERMINISTIC_FALLBACK
    }

    /**
     * Lightweight Read-Model projection of an Engagement File's template version state.
     * Decouples dashboard reads from the ~1-minute Engagement File load constraint.
     */
    public record EngagementMetadata(
            UUID engagementId,
            UUID firmId,
            String templateId,
            int appliedVersion,
            Integer declinedVersion,
            EngagementStatus status,
            long eventSequence
    ) {
        public EngagementMetadata {
            Objects.requireNonNull(engagementId, "engagementId must not be null");
            Objects.requireNonNull(firmId, "firmId must not be null");
            Objects.requireNonNull(templateId, "templateId must not be null");
            Objects.requireNonNull(status, "status must not be null");
            if (appliedVersion < 1) {
                throw new IllegalArgumentException("appliedVersion must be >= 1");
            }
            if (declinedVersion != null && declinedVersion <= appliedVersion) {
                throw new IllegalArgumentException("declinedVersion must be greater than appliedVersion");
            }
        }

        public OptionalInt optionalDeclinedVersion() {
            return declinedVersion == null ? OptionalInt.empty() : OptionalInt.of(declinedVersion);
        }

        /**
         * Determines whether a template update should be surfaced to the user.
         * True when latestVersion > appliedVersion AND (not declined OR latestVersion > declinedVersion).
         */
        public boolean hasPendingUpdate(int latestTemplateVersion) {
            return latestTemplateVersion > appliedVersion
                    && (declinedVersion == null || latestTemplateVersion > declinedVersion);
        }

        public EngagementMetadata markApplyInProgress(long newEventSequence) {
            if (this.status == EngagementStatus.APPLY_IN_PROGRESS) {
                throw new ConcurrentEngagementMutationException(
```

```
                    "Engagement " + engagementId + " already has a template apply operation in progress.");
        }
        return new EngagementMetadata(
                engagementId, firmId, templateId, appliedVersion, declinedVersion,
                EngagementStatus.APPLY_IN_PROGRESS, newEventSequence
        );
    }

    public EngagementMetadata completeApply(int newAppliedVersion, long newEventSequence) {
        if (newEventSequence <= this.eventSequence) {
            return this; // Idempotent ignore of stale/out-of-order events
        }
        if (newAppliedVersion <= this.appliedVersion) {
            throw new IllegalArgumentException("New applied version must advance current appliedVersion");
        }
        // Applying a newer version clears any prior declinedVersion marker
        return new EngagementMetadata(
                engagementId, firmId, templateId, newAppliedVersion, null,
                EngagementStatus.READY, newEventSequence
        );
    }

    public EngagementMetadata recordDecline(int versionToDecline, long newEventSequence) {
        if (this.status == EngagementStatus.APPLY_IN_PROGRESS) {
            throw new ConcurrentEngagementMutationException(
                    "Cannot decline an update while an apply operation is in progress for engagement " + engagementI
        }
        if (newEventSequence <= this.eventSequence) {
            return this; // Idempotent ignore of stale/out-of-order events
        }
        if (versionToDecline <= this.appliedVersion) {
            throw new IllegalArgumentException("Cannot decline a version <= appliedVersion");
        }
        return new EngagementMetadata(
                engagementId, firmId, templateId, appliedVersion, versionToDecline,
                EngagementStatus.READY, newEventSequence
        );
    }
}

/**
 * Represents a deterministic change within a single logical section of a Product Template JSON archive.
 */
public record SectionChange(
        String sectionId,
        String sectionTitle,
        ChangeType changeType,
        List<String> modifiedJsonPaths
) {
    public SectionChange {
        Objects.requireNonNull(sectionId, "sectionId must not be null");
        Objects.requireNonNull(sectionTitle, "sectionTitle must not be null");
        Objects.requireNonNull(changeType, "changeType must not be null");
        modifiedJsonPaths = List.copyOf(Objects.requireNonNull(modifiedJsonPaths, "modifiedJsonPaths must not be nul
        if (modifiedJsonPaths.isEmpty()) {
            throw new IllegalArgumentException("SectionChange must contain at least one modified JSON path");
        }
    }
}

/**
 * Deterministic net JSON diff between two versions (fromVersion -> toVersion) of a Product Template.
 */
public record TemplateDiff(
        String templateId,
        int fromVersion,
        int toVersion,
        List<SectionChange> sectionChanges
) {
    public TemplateDiff {
        Objects.requireNonNull(templateId, "templateId must not be null");
        if (fromVersion < 1 || toVersion <= fromVersion) {
            throw new IllegalArgumentException("Invalid version range: " + fromVersion + " -> " + toVersion);
        }
```

```
        sectionChanges = List.copyOf(Objects.requireNonNull(sectionChanges, "sectionChanges must not be null"));
    }

    public Set<String> allSectionIds() {
        return sectionChanges.stream()
                .map(SectionChange::sectionId)
                .collect(Collectors.toUnmodifiableSet());
    }

    public Set<String> allJsonPaths() {
        return sectionChanges.stream()
                .flatMap(sc -> sc.modifiedJsonPaths().stream())
                .collect(Collectors.toUnmodifiableSet());
    }

    public Set<String> jsonPathsForSection(String sectionId) {
        return sectionChanges.stream()
                .filter(sc -> sc.sectionId().equals(sectionId))
                .flatMap(sc -> sc.modifiedJsonPaths().stream())
                .collect(Collectors.toUnmodifiableSet());
    }
}

/**
 * Structured item returned by the LLM for a specific template section.
 */
public record SectionSummaryItem(
        String sectionId,
        String humanReadableSummary,
        List<String> citedJsonPaths
) {
    public SectionSummaryItem {
        Objects.requireNonNull(sectionId, "sectionId must not be null");
        Objects.requireNonNull(humanReadableSummary, "humanReadableSummary must not be null");
        citedJsonPaths = List.copyOf(Objects.requireNonNull(citedJsonPaths, "citedJsonPaths must not be null"));
    }
}

/**
 * Raw structured response produced by the LLM before deterministic validation.
 */
public record LlmDraftSummary(
        String executiveSummary,
        List<SectionSummaryItem> sectionItems
) {
    public LlmDraftSummary {
        Objects.requireNonNull(executiveSummary, "executiveSummary must not be null");
        sectionItems = List.copyOf(Objects.requireNonNull(sectionItems, "sectionItems must not be null"));
    }
}

/**
 * Final verified summary result delivered to the API / Read Model cache.
 * Guaranteed to be either 100% citation-validated AI output or a deterministic structural fallback.
 */
public record AiSummaryResult(
        String templateId,
        int fromVersion,
        int toVersion,
        SummarySource source,
        String executiveSummary,
        List<SectionSummaryItem> sectionSummaries,
        List<String> validationWarnings
) {
    public AiSummaryResult {
        Objects.requireNonNull(templateId, "templateId must not be null");
        Objects.requireNonNull(source, "source must not be null");
        Objects.requireNonNull(executiveSummary, "executiveSummary must not be null");
        sectionSummaries = List.copyOf(Objects.requireNonNull(sectionSummaries, "sectionSummaries must not be null")
        validationWarnings = List.copyOf(Objects.requireNonNull(validationWarnings, "validationWarnings must not be
    }
}

/**
```

```
     * View returned to the practitioner inspecting a pending update on an engagement.
     */
    public record PendingUpdateEvaluationView(
            UUID engagementId,
            String templateId,
            int currentlyAppliedVersion,
            Integer previouslyDeclinedVersion,
            int targetLatestVersion,
            AiSummaryResult cumulativeSummary
    ) {}

    public static class ConcurrentEngagementMutationException extends RuntimeException {
        public ConcurrentEngagementMutationException(String message) {
            super(message);
        }
    }

    public static class LlmSummarizationException extends RuntimeException {
        public LlmSummarizationException(String message, Throwable cause) {
            super(message, cause);
        }
        public LlmSummarizationException(String message) {
            super(message);
        }
    }
}
```

2. Port Interfaces (`Ports.java`)

```
package com.caseware.updates;

import com.caseware.updates.DomainModels.LlmDraftSummary;
import com.caseware.updates.DomainModels.TemplateDiff;

/**
 * Port interface for the existing deterministic Product Template JSON Diff tool.
 */
interface TemplateDiffEngine {
    /**
     * Extracts the exact net JSON diff between fromVersion and toVersion.
     */
    TemplateDiff computeNetDiff(String templateId, int fromVersion, int toVersion);
}

/**
 * Port interface for the LLM summarization adapter.
 */
interface LlmSummarizer {
    /**
     * Translates a deterministic TemplateDiff into a structured, human-readable draft summary.
     */
    LlmDraftSummary summarizeDiff(TemplateDiff diff);
}
```

3. Deterministic AI Validator & Orchestrator (`SummaryCitationValidator.java`)

Enforces two strict audit guardrails on every LLM response:

1. **Zero Hallucinated Paths/Sections:** Every `sectionId` and `citedJsonPath` in the LLM output must exist within that exact section of the `TemplateDiff`.
2. **100% Section Coverage:** Every modified `sectionId` in the `TemplateDiff` must be represented in the LLM output. If validation fails or the LLM throws a timeout/runtime exception, `SummarizationOrchestrator` automatically falls back to a deterministic human-formatted table of the structural diff headers.

```
package com.caseware.updates;

import com.caseware.updates.DomainModels.*;
import java.util.ArrayList;
import java.util.List;
import java.util.Objects;
import java.util.Set;
import java.util.stream.Collectors;
```

```
/**
 * Deterministic Guardrail: Validates that LLM output cites only real JSON paths from the diff
 * and covers 100% of modified template sections without omission.
 */
public final class SummaryCitationValidator {

    public record ValidationResult(boolean isValid, List<String> violations) {
        public ValidationResult {
            violations = List.copyOf(violations);
        }
    }

    public ValidationResult validate(TemplateDiff diff, LlmDraftSummary draft) {
        Objects.requireNonNull(diff, "diff must not be null");
        if (draft == null) {
            return new ValidationResult(false, List.of("LLM draft summary is null"));
        }

        List<String> violations = new ArrayList<>();

        if (draft.executiveSummary().isBlank()) {
            violations.add("Executive summary is blank");
        }

        Set<String> expectedSections = diff.allSectionIds();
        Set<String> citedSections = draft.sectionItems().stream()
                .map(SectionSummaryItem::sectionId)
                .collect(Collectors.toSet());

        // 1. Check for hallucinated sections not present in the raw JSON diff
        for (SectionSummaryItem item : draft.sectionItems()) {
            if (!expectedSections.contains(item.sectionId())) {
                violations.add("Hallucinated sectionId not in diff: " + item.sectionId());
                continue;
            }
            if (item.humanReadableSummary().isBlank()) {
                violations.add("Blank summary for section: " + item.sectionId());
            }
            if (item.citedJsonPaths().isEmpty()) {
                violations.add("No JSON paths cited for section: " + item.sectionId());
            }

            // 2. Check that every cited JSON path actually belongs to that section in the raw diff
            Set<String> validPathsForSection = diff.jsonPathsForSection(item.sectionId());
            for (String citedPath : item.citedJsonPaths()) {
                if (!validPathsForSection.contains(citedPath)) {
                    violations.add("Hallucinated or misattributed JSON path '" + citedPath
                            + "' in section '" + item.sectionId() + "'");
                }
            }
        }

        // 3. Check 100% section coverage (no silent omissions of compliance-critical sections)
        for (String expectedSection : expectedSections) {
            if (!citedSections.contains(expectedSection)) {
                violations.add("Missing coverage for modified section: " + expectedSection);
            }
        }

        return new ValidationResult(violations.isEmpty(), violations);
    }
}

/**
 * Orchestrates net-diff extraction, LLM summarization, citation validation,
 * and automatic deterministic fallback when LLM validation fails or times out.
 */
final class SummarizationOrchestrator {

    private final TemplateDiffEngine diffEngine;
    private final LlmSummarizer llmSummarizer;
    private final SummaryCitationValidator validator;

    public SummarizationOrchestrator(
```

```
            TemplateDiffEngine diffEngine,
            LlmSummarizer llmSummarizer,
            SummaryCitationValidator validator
    ) {
        this.diffEngine = Objects.requireNonNull(diffEngine);
        this.llmSummarizer = Objects.requireNonNull(llmSummarizer);
        this.validator = Objects.requireNonNull(validator);
    }

    /**
     * Computes a human-readable summary between fromVersion and toVersion.
     * Always computes the direct net JSON diff (fromVersion -> toVersion) rather than chaining text summaries.
     */
    public AiSummaryResult generateVerifiedSummary(String templateId, int fromVersion, int toVersion) {
        TemplateDiff netDiff = diffEngine.computeNetDiff(templateId, fromVersion, toVersion);

        try {
            LlmDraftSummary draft = llmSummarizer.summarizeDiff(netDiff);
            SummaryCitationValidator.ValidationResult validation = validator.validate(netDiff, draft);

            if (validation.isValid()) {
                return new AiSummaryResult(
                        templateId,
                        fromVersion,
                        toVersion,
                        SummarySource.AI_VALIDATED,
                        draft.executiveSummary(),
                        draft.sectionItems(),
                        List.of()
                );
            }
            return buildDeterministicFallback(netDiff, validation.violations());
        } catch (Exception ex) {
            return buildDeterministicFallback(
                    netDiff,
                    List.of("LLM summarization failed or timed out: " + ex.getMessage())
            );
        }
    }

    private AiSummaryResult buildDeterministicFallback(TemplateDiff diff, List<String> warnings) {
        String execSummary = String.format(
                "Structured Change Overview (v%d to v%d): %d template section(s) modified.",
                diff.fromVersion(), diff.toVersion(), diff.sectionChanges().size()
        );

        List<SectionSummaryItem> fallbackItems = diff.sectionChanges().stream()
                .map(sc -> new SectionSummaryItem(
                        sc.sectionId(),
                        String.format("[%s] %s (%d field path(s) updated)",
                                sc.changeType().name(), sc.sectionTitle(), sc.modifiedJsonPaths().size()),
                        sc.modifiedJsonPaths()
                ))
                .toList();

        return new AiSummaryResult(
                diff.templateId(),
                diff.fromVersion(),
                diff.toVersion(),
                SummarySource.DETERMINISTIC_FALLBACK,
                execSummary,
                fallbackItems,
                warnings
        );
    }
}
```

4. Engagement Update Domain Service (`EngagementUpdateDomainService.java`)

Manages pending update detection, template-level caching of `(templateId, fromVersion, toVersion)` summaries across engagements, direct net-diff evaluation across declined versions (`v1 -> v3`), and `APPLY_IN_PROGRESS` concurrency locking.

```
package com.caseware.updates;
```

```
import com.caseware.updates.DomainModels.*;
import java.util.Map;
import java.util.Objects;
import java.util.UUID;
import java.util.concurrent.ConcurrentHashMap;

/**
 * Domain service managing Engagement update checks, cumulative net-diff evaluation across
 * accumulated or declined versions, and concurrency locking during the 1-minute apply window.
 */
public final class EngagementUpdateDomainService {

    public record SummaryCacheKey(String templateId, int fromVersion, int toVersion) {}

    private final Map<UUID, EngagementMetadata> engagementStore = new ConcurrentHashMap<>();
    private final Map<String, Integer> latestTemplateVersions = new ConcurrentHashMap<>();
    private final Map<SummaryCacheKey, AiSummaryResult> summaryCache = new ConcurrentHashMap<>();
    private final SummarizationOrchestrator orchestrator;

    public EngagementUpdateDomainService(SummarizationOrchestrator orchestrator) {
        this.orchestrator = Objects.requireNonNull(orchestrator);
    }

    public void registerEngagement(EngagementMetadata metadata) {
        engagementStore.put(metadata.engagementId(), metadata);
    }

    public void publishTemplateVersion(String templateId, int newVersion) {
        latestTemplateVersions.merge(templateId, newVersion, Math::max);
    }

    public EngagementMetadata getEngagement(UUID engagementId) {
        EngagementMetadata meta = engagementStore.get(engagementId);
        if (meta == null) {
            throw new IllegalArgumentException("Unknown engagementId: " + engagementId);
        }
        return meta;
    }

    public boolean hasPendingUpdate(UUID engagementId) {
        EngagementMetadata meta = getEngagement(engagementId);
        int latestVersion = latestTemplateVersions.getOrDefault(meta.templateId(), meta.appliedVersion());
        return meta.hasPendingUpdate(latestVersion);
    }

    /**
     * Evaluates the pending inbound update for an engagement.
     * Always summarizes the cumulative net diff (appliedVersion -> latestVersion), even if
     * intermediate versions were declined.
     */
    public PendingUpdateEvaluationView evaluatePendingUpdate(UUID engagementId) {
        EngagementMetadata meta = getEngagement(engagementId);
        int latestVersion = latestTemplateVersions.getOrDefault(meta.templateId(), meta.appliedVersion());

        if (!meta.hasPendingUpdate(latestVersion)) {
            throw new IllegalStateException("No pending update to evaluate for engagement " + engagementId);
        }

        SummaryCacheKey cacheKey = new SummaryCacheKey(meta.templateId(), meta.appliedVersion(), latestVersion);
        AiSummaryResult summary = summaryCache.computeIfAbsent(cacheKey, key ->
                orchestrator.generateVerifiedSummary(key.templateId(), key.fromVersion(), key.toVersion())
        );

        return new PendingUpdateEvaluationView(
                meta.engagementId(),
                meta.templateId(),
                meta.appliedVersion(),
                meta.declinedVersion(),
                latestVersion,
                summary
        );
    }

    /**
```

```
     * Acquires the APPLY_IN_PROGRESS lock before the Engagement Management System begins
     * the ~1-minute file load and template application process.
     */
    public EngagementMetadata initiateApply(UUID engagementId, long eventSequence) {
        return engagementStore.compute(engagementId, (id, current) -> {
            if (current == null) {
                throw new IllegalArgumentException("Unknown engagementId: " + id);
            }
            return current.markApplyInProgress(eventSequence);
        });
    }

    /**
     * Handles the completion event once the Engagement Management System finishes applying the template.
     */
    public EngagementMetadata handleUpdateAppliedEvent(UUID engagementId, int newAppliedVersion, long eventSequence) {
        return engagementStore.compute(engagementId, (id, current) -> {
            if (current == null) {
                throw new IllegalArgumentException("Unknown engagementId: " + id);
            }
            return current.completeApply(newAppliedVersion, eventSequence);
        });
    }

    /**
     * Records a user's decision to decline a specific template version.
     * The engagement stays on its appliedVersion, suppresses notifications for declinedVersion,
     * and will re-notify when a version > declinedVersion is published.
     */
    public EngagementMetadata declineUpdate(UUID engagementId, int versionToDecline, long eventSequence) {
        return engagementStore.compute(engagementId, (id, current) -> {
            if (current == null) {
                throw new IllegalArgumentException("Unknown engagementId: " + id);
            }
            return current.recordDecline(versionToDecline, eventSequence);
        });
    }
}
```

5. JUnit 5 Verification Suite (`EngagementUpdateSystemTest.java`)

Covers all six required behavioral and failure-mode scenarios:

1. Valid AI summary acceptance when 100% of sections and JSON paths match the diff.
2. Rejection & deterministic fallback when the LLM hallucinates or misattributes a JSON path.
3. Rejection & deterministic fallback when the LLM omits a modified section.
4. Graceful fallback when the LLM times out.
5. Accumulated updates with an intermediate decline (`v1 -> v2 declined -> v3`), proving re-notification on `v3` and verifying that `TemplateDiffEngine` is called directly with `(1, 3)` rather than `(2, 3)`.
6. Concurrency locking (`APPLY_IN_PROGRESS`) during the 1-minute engagement load window and idempotent handling of out-of-order events.

```
package com.caseware.updates;

import com.caseware.updates.DomainModels.*;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;

import java.util.ArrayList;
import java.util.List;
import java.util.UUID;

import static org.junit.jupiter.api.Assertions.*;

class EngagementUpdateSystemTest {

    private static final String TEMPLATE_ID = "audit-ifrs-2026";
    private UUID firmId;
    private UUID engagementId;
    private SummaryCitationValidator validator;
```

```
    @BeforeEach
    void setUp() {
        firmId = UUID.randomUUID();
        engagementId = UUID.randomUUID();
        validator = new SummaryCitationValidator();
    }

    private TemplateDiff sampleDiffV1ToV3() {
        return new TemplateDiff(
                TEMPLATE_ID,
                1,
                3,
                List.of(
                        new SectionChange(
                                "sec-rev-01",
                                "Revenue Recognition Checklist",
                                ChangeType.MODIFIED,
                                List.of("/sections/sec-rev-01/materialityThreshold", "/sections/sec-rev-01/ifrs15Step3")
                        ),
                        new SectionChange(
                                "sec-esg-02",
                                "Climate Risk Disclosures",
                                ChangeType.ADDED,
                                List.of("/sections/sec-esg-02/scope1Emissions")
                        )
                )
        );
    }

    @Test
    @DisplayName("1. Validator accepts AI summary when 100% of modified sections and valid JSON paths are cited")
    void shouldAcceptValidAiSummary() {
        TemplateDiff diff = sampleDiffV1ToV3();
        LlmDraftSummary validDraft = new LlmDraftSummary(
                "Updated revenue materiality rules and added Scope 1 climate disclosures.",
                List.of(
                        new SectionSummaryItem(
                                "sec-rev-01",
                                "Adjusted materiality threshold and clarified IFRS 15 Step 3 guidance.",
                                List.of("/sections/sec-rev-01/materialityThreshold", "/sections/sec-rev-01/ifrs15Step3")
                        ),
                        new SectionSummaryItem(
                                "sec-esg-02",
                                "Added mandatory Scope 1 emissions disclosure checklist.",
                                List.of("/sections/sec-esg-02/scope1Emissions")
                        )
                )
        );

        SummarizationOrchestrator orchestrator = new SummarizationOrchestrator(
                (t, f, to) -> diff,
                d -> validDraft,
                validator
        );

        AiSummaryResult result = orchestrator.generateVerifiedSummary(TEMPLATE_ID, 1, 3);

        assertEquals(SummarySource.AI_VALIDATED, result.source());
        assertTrue(result.validationWarnings().isEmpty());
        assertEquals(2, result.sectionSummaries().size());
    }

    @Test
    @DisplayName("2. Validator rejects hallucinated JSON paths and degrades to deterministic structural diff")
    void shouldFallbackWhenLlmHallucinatesJsonPath() {
        TemplateDiff diff = sampleDiffV1ToV3();
        LlmDraftSummary hallucinatedDraft = new LlmDraftSummary(
                "Summary with fabricated path.",
                List.of(
                        new SectionSummaryItem(
                                "sec-rev-01",
                                "Updated revenue rules.",
                                List.of("/sections/sec-rev-01/nonExistentHallucinatedField")
```

```
                    ),
                    new SectionSummaryItem(
                            "sec-esg-02",
                            "Added climate disclosures.",
                            List.of("/sections/sec-esg-02/scope1Emissions")
                    )
            )
    );

    SummarizationOrchestrator orchestrator = new SummarizationOrchestrator(
            (t, f, to) -> diff,
            d -> hallucinatedDraft,
            validator
    );

    AiSummaryResult result = orchestrator.generateVerifiedSummary(TEMPLATE_ID, 1, 3);

    assertEquals(SummarySource.DETERMINISTIC_FALLBACK, result.source());
    assertFalse(result.validationWarnings().isEmpty());
    assertTrue(result.validationWarnings().getFirst().contains("Hallucinated or misattributed JSON path"));
    assertEquals(2, result.sectionSummaries().size());
    assertTrue(result.sectionSummaries().getFirst().humanReadableSummary().contains("[MODIFIED] Revenue Recognition
}

@Test
@DisplayName("3. Validator rejects partial section coverage (omitted section) and degrades to fallback")
void shouldFallbackWhenLlmOmitsModifiedSection() {
    TemplateDiff diff = sampleDiffV1ToV3();
    // Draft omits sec-esg-02 completely
    LlmDraftSummary incompleteDraft = new LlmDraftSummary(
            "Partial summary.",
            List.of(
                    new SectionSummaryItem(
                            "sec-rev-01",
                            "Updated revenue rules.",
                            List.of("/sections/sec-rev-01/materialityThreshold")
                    )
            )
    );

    SummarizationOrchestrator orchestrator = new SummarizationOrchestrator(
            (t, f, to) -> diff,
            d -> incompleteDraft,
            validator
    );

    AiSummaryResult result = orchestrator.generateVerifiedSummary(TEMPLATE_ID, 1, 3);

    assertEquals(SummarySource.DETERMINISTIC_FALLBACK, result.source());
    assertTrue(result.validationWarnings().stream()
            .anyMatch(w -> w.contains("Missing coverage for modified section: sec-esg-02")));
}

@Test
@DisplayName("4. Orchestrator degrades gracefully to deterministic summary when LLM times out")
void shouldFallbackOnLlmTimeout() {
    TemplateDiff diff = sampleDiffV1ToV3();
    SummarizationOrchestrator orchestrator = new SummarizationOrchestrator(
            (t, f, to) -> diff,
            d -> { throw new LlmSummarizationException("AWS Bedrock invocation timed out after 5000ms"); },
            validator
    );

    AiSummaryResult result = orchestrator.generateVerifiedSummary(TEMPLATE_ID, 1, 3);

    assertEquals(SummarySource.DETERMINISTIC_FALLBACK, result.source());
    assertEquals(2, result.sectionSummaries().size());
    assertTrue(result.validationWarnings().getFirst().contains("timed out"));
}

@Test
@DisplayName("5. Accumulated updates (v1 -> v2 declined -> v3): re-notifies on v3 and computes net diff (v1 -> v3)")
void shouldHandleDeclinedVersionAndComputeCumulativeNetDiffOnNextVersion() {
    List<String> requestedDiffPairs = new ArrayList<>();
```

```
    TemplateDiffEngine trackingDiffEngine = (templateId, fromVersion, toVersion) -> {
        requestedDiffPairs.add(fromVersion + "->" + toVersion);
        return sampleDiffV1ToV3();
    };

    LlmSummarizer validSummarizer = diff -> new LlmDraftSummary(
            "Cumulative v1 to v3 summary",
            diff.sectionChanges().stream()
                    .map(sc -> new SectionSummaryItem(sc.sectionId(), "Updated " + sc.sectionTitle(), sc.modifiedJso
                    .toList()
    );

    SummarizationOrchestrator orchestrator = new SummarizationOrchestrator(trackingDiffEngine, validSummarizer, vali
    EngagementUpdateDomainService service = new EngagementUpdateDomainService(orchestrator);

    // Step 1: Engagement created on v1
    service.registerEngagement(new EngagementMetadata(
            engagementId, firmId, TEMPLATE_ID, 1, null, EngagementStatus.READY, 1L
    ));
    service.publishTemplateVersion(TEMPLATE_ID, 1);
    assertFalse(service.hasPendingUpdate(engagementId));

    // Step 2: v2 is published -> engagement has pending update
    service.publishTemplateVersion(TEMPLATE_ID, 2);
    assertTrue(service.hasPendingUpdate(engagementId));

    // Step 3: User declines v2 -> engagement stays on appliedVersion=1, declinedVersion=2; notification suppressed
    service.declineUpdate(engagementId, 2, 2L);
    assertFalse(service.hasPendingUpdate(engagementId));
    assertEquals(1, service.getEngagement(engagementId).appliedVersion());
    assertEquals(2, service.getEngagement(engagementId).declinedVersion());

    // Step 4: v3 is published -> user is notified again because v3 > declinedVersion (2)
    service.publishTemplateVersion(TEMPLATE_ID, 3);
    assertTrue(service.hasPendingUpdate(engagementId));

    // Step 5: Evaluating pending update on v3 MUST compute direct net diff (1 -> 3), NOT (2 -> 3)
    PendingUpdateEvaluationView view = service.evaluatePendingUpdate(engagementId);
    assertEquals(1, view.currentlyAppliedVersion());
    assertEquals(2, view.previouslyDeclinedVersion());
    assertEquals(3, view.targetLatestVersion());
    assertEquals(List.of("1->3"), requestedDiffPairs);
    assertEquals(SummarySource.AI_VALIDATED, view.cumulativeSummary().source());
}

@Test
@DisplayName("6. Concurrency lock (APPLY_IN_PROGRESS) blocks overlapping mutations and ignores stale events")
void shouldPreventConcurrentMutationsDuringOneMinuteApplyWindow() {
    SummarizationOrchestrator orchestrator = new SummarizationOrchestrator(
            (t, f, to) -> sampleDiffV1ToV3(),
            d -> new LlmDraftSummary("Summary", List.of()),
            validator
    );
    EngagementUpdateDomainService service = new EngagementUpdateDomainService(orchestrator);

    service.registerEngagement(new EngagementMetadata(
            engagementId, firmId, TEMPLATE_ID, 1, null, EngagementStatus.READY, 10L
    ));
    service.publishTemplateVersion(TEMPLATE_ID, 2);

    // User A clicks Apply -> locks engagement in APPLY_IN_PROGRESS
    EngagementMetadata locked = service.initiateApply(engagementId, 11L);
    assertEquals(EngagementStatus.APPLY_IN_PROGRESS, locked.status());

    // User B attempts to click Apply or Decline while file is still loading (~1 min window) -> rejected
    assertThrows(ConcurrentEngagementMutationException.class,
            () -> service.initiateApply(engagementId, 12L));
    assertThrows(ConcurrentEngagementMutationException.class,
            () -> service.declineUpdate(engagementId, 2, 12L));

    // Apply finishes -> transitions to v2, status READY, clears any declinedVersion
    EngagementMetadata completed = service.handleUpdateAppliedEvent(engagementId, 2, 13L);
    assertEquals(EngagementStatus.READY, completed.status());
```

```
        assertEquals(2, completed.appliedVersion());
        assertNull(completed.declinedVersion());

        // Out-of-order / duplicate event with older sequence number (e.g., seq 12L) is safely ignored
        EngagementMetadata afterStaleEvent = service.handleUpdateAppliedEvent(engagementId, 1, 12L);
        assertEquals(2, afterStaleEvent.appliedVersion());
        assertEquals(13L, afterStaleEvent.eventSequence());
    }
}
```

Would you also like to prepare a short **AI Usage & Live Design Review Cheat Sheet** covering the four follow-up review questions from the prompt (where AI helped, where you corrected AI output, how to guide other engineers, and where AI should never be trusted in this domain)?

User prompt: The tests pass, but I don't like how the project structure looks in IntelliJ. Nesting all records and exceptions inside `DomainModels.java` and putting multiple package-private classes/interfaces inside `Ports.java` and `SummaryCitationValidator.java` feels unnatural and hard to read for a human developer. Please refactor the code into a clean, standard Java layered package structure (`model`, `client`, `validator`, `service`, `exception`) where every public class, interface, record, and enum lives in its own `.java` file.

Response: Your ZIP file is ready

[file-tag: code-generated-file-81a2fa87-3710-4b21-a1cc-354c1fa069c8]

The project has been refactored into a standard layered Java package structure (`client`, `exception`, `model`, `service`, `validator`) where every `public` class, interface, record, and enum resides in its own dedicated `.java` file.

Refactored Project Layout

```
caseware-template-update-slice/
├── pom.xml
└── src
    ├── main/java/com/caseware/updates
    │   ├── client
    │   │   ├── LlmSummarizer.java
    │   │   └── TemplateDiffEngine.java
    │   ├── exception
    │   │   ├── ConcurrentEngagementMutationException.java
    │   │   └── LlmSummarizationException.java
    │   ├── model
    │   │   ├── AiSummaryResult.java
    │   │   ├── ChangeType.java
    │   │   ├── EngagementMetadata.java
    │   │   ├── EngagementStatus.java
    │   │   ├── LlmDraftSummary.java
    │   │   ├── PendingUpdateEvaluationView.java
    │   │   ├── SectionChange.java
    │   │   ├── SectionSummaryItem.java
    │   │   ├── SummaryCacheKey.java
    │   │   ├── SummarySource.java
    │   │   └── TemplateDiff.java
    │   ├── service
    │   │   ├── EngagementUpdateDomainService.java
    │   │   └── SummarizationOrchestrator.java
    │   └── validator
    │       ├── SummaryCitationValidator.java
    │       └── ValidationResult.java
    └── test/java/com/caseware/updates
        └── EngagementUpdateSystemTest.java
```

What Changed in `v2`

1. **Unnested Domain Model (`com.caseware.updates.model`):**
    - Replaced the monolithic `DomainModels` wrapper class with 11 standalone `public` records and enums (`EngagementMetadata`, `TemplateDiff`, `SectionChange`, `SectionSummaryItem`, `LlmDraftSummary`, `AiSummaryResult`, `PendingUpdateEvaluationView`, `SummaryCacheKey`, `EngagementStatus`, `ChangeType`, `SummarySource`).
2. **Explicit Port Contracts (`com.caseware.updates.client`):**
    - Extracted `TemplateDiffEngine` and `LlmSummarizer` from the package-private `Ports.java` container into standalone `public` interfaces.
3. **Separated Validator & Orchestrator (`com.caseware.updates.validator` & `com.caseware.updates.service`):**

- Extracted `ValidationResult` and `SummaryCitationValidator` into `validator`, and moved `SummarizationOrchestrator` alongside `EngagementUpdateDomainService` in `service`.

4. **Dedicated Domain Exceptions (`com.caseware.updates.exception`):**
   - Moved `ConcurrentEngagementMutationException` and `LlmSummarizationException` out of static inner classes into top-level exception classes.

Tip for Your Live Design Review ("Where You Corrected AI Output")

The assessment specifically notes that the panel will ask **"Where you corrected or ignored AI output"** during the follow-up review. You now have three concrete, genuine examples from this session to highlight:

1. **Preventing Chained Summary Hallucinations:** Rejecting the idea of having an LLM combine sequential text summaries (`v1->v2` + `v2->v3`) and enforcing direct net JSON diffing (`v1->v3`) so intermediate reverts cancel out deterministically.
2. **Enforcing Strict Audit Guardrails:** Requiring 100% section coverage and exact JSON-path citation validation with a deterministic structural fallback on failure or timeout.
3. **Codebase Maintainability & Packaging Standards:** Rejecting the AI's compact single-file inner-class shortcuts (`DomainModels.java`, `Ports.java`) and enforcing idiomatic, single-responsibility Java package boundaries (`model`, `client`, `validator`, `service`, `exception`).
